DATA STRUCTURES AND ALGORITHMS · LAB WORK

B-trees

Search, insertion with node splitting, and deletion with borrowing and merging

A B-tree is a balanced multiway search tree: every node holds several ordered keys and has one more child than it has keys. Bayer and McCreight introduced it for indexes stored on disk [1]. A node fills one page, and the tree stays so shallow that a search reads only a few pages [2]. This lab follows the variant used in the course examples: the B-tree of order d = 2, where an insertion splits full nodes and a deletion borrows a key from a sibling or merges two nodes.

Contents

1 Objectives . . . 2

2 Background . . . 2

2.1 The properties of a B-tree . . . 2

2.2 The NodeB structure . . . 3

2.3 The height of the tree . . . 3

3 B-tree operations . . . 3

3.1 Search . . . 3

3.2 Insertion . . . 3

3.3 Deletion . . . 6

3.4 Listing . . . 10

3.5 Cost of the operations . . . 10

4 Lab work . . . 10

4.1 The complete functions . . . 11

4.2 Expected results . . . 16

5 Exercises . . . 16

6 Practice questions . . . 17

7 Review questions . . . 17

8 Source code . . . 18

9 References . . . 26

1 Objectives

- Understanding the properties of a B-tree and how the order, the number of keys in a node and the height of the tree relate.
- Following step by step the search, the insertion that splits full nodes and the deletion that borrows and merges.
- Implementing these operations in C++ on the node structure of the course, with the arrays `key[]` and `child[]`.
- Checking the properties of the tree after every operation.
- Analysing the cost of the operations in terms of the height and the order of the tree.

2 Background

2.1 The properties of a B-tree

A **B-tree of order d** has the following properties [1, 4]:

- every node holds at most 2d keys, and an internal node with k keys has exactly k + 1 children;
- every node except the root holds at least d keys; the root holds at least one key;
- all leaves are on the same level;
- the keys of a node are in increasing order, and the subtree `child[i]` holds only keys between `key[i]` and `key[i + 1]`.

Sources do not define the order the same way. In the course, as in Knuth [3], the order m is the largest number of children of a node. Bayer and McCreight [1] and Wirth [4] call the smallest number of keys, d, the order, and Cormen et al. [5] use the minimum degree t, with at most $2t - 1$ keys in a node. The course examples have nodes with at most 4 keys, so d = 2, that is m = 5. The 2-3 tree is the case d = 1. A full node in [5] always holds an odd number of keys, so the trees of the course examples cannot be described with a minimum degree t. Figure 1 shows the tree used as an example throughout this lab.

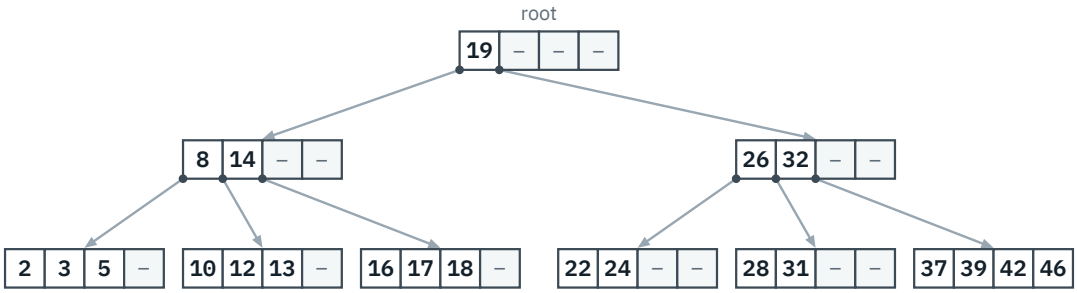


Figure 1. The B-tree of order d = 2 from the deletion example of the course. Every node has 2d = 4 key cells, and free cells are marked "–". The arrows leave from between the keys: `child[i]` leads to the keys between `key[i]` and `key[i + 1]`. The tree is built by inserting, in this order, the keys `2, 5, 8, 10, 12, 14, 16, 17, 26, 28, 31, 32, 37, 39, 22, 24, 19, 18, 3, 13, 42, 46`.

2.2 The NodeB structure

In C++ every node uses the same structure, `NodeB`, described in Table 1. As in the course, keys are numbered from 1 and children from 0.

Table 1. The fields of `NodeB`.

FIELD	TYPE	ROLE
`nr`	`int`	the number of keys in the node
`key[2d + 2]`	`int`	the keys `key[1..nr]`, in increasing order; `key[0]` is unused, and `key[2d + 1]` briefly holds the extra key until the node is split
`child[2d + 2]`	`NodeB*`	the children `child[0..nr]`; `child[i]` holds the keys between `key[i]` and `key[i + 1]`; in a leaf every pointer is `nullptr`

On a 64-bit machine, with `D = 2`, a node takes 80 bytes with g++ (`sizeof(NodeB)`): 4 for `nr`, 24 for the 6 keys, 4 of padding and 48 for the 6 pointers.

2.3 The height of the tree

The root holds at least one key and every other node at least d keys. A tree of height h therefore holds at least $2(d + 1)^h - 1$ keys and at most $(2d + 1)^{h+1} - 1$ keys. It follows that:

$$h \le \log_{d+1}((n + 1) / 2)$$

For $n = 10^6$ keys, a tree with $d = 2$ has height at most 11. With $d = 50$, a node the size of a disk page of a few kilobytes, the height is at most 3, so any key is found by reading at most 4 pages.

3 B-tree operations

3.1 Search

In the current node, find the first position `i` with `k ≤ key[i]` (the function `position`). If `key[i] = k`, the key is found. Otherwise the only subtree that can hold it is `child[i − 1]`, with the keys between `key[i − 1]` and `key[i]`. In a leaf this pointer is null, so the key is not there. A search visits at most h + 1 nodes.

In Figure 1, the search for 24 passes 19 in the root and goes down through `child[1]`, then, in node `[26 32]`, through `child[0]`, since 24 < 26, and finds the key in the leaf `[22 24]`. The search for 11 goes down through `child[0]` of the root and `child[1]` of node `[8 14]`, to the leaf `[10 12 13]`. There 10 < 11 < 12, and `child[1]` is null, so the key is not there.

3.2 Insertion

Go down as in a search to the leaf where the key belongs, shift the larger keys one position to the right and put `k` at position `i`. If the node now holds 2d + 1 keys, it is split: the first d keys stay, the median `key[d + 1]` moves up to the parent, and the last d keys move to a new node, which becomes the right child of the median. The parent may become full in turn and is split, and so on. When the root is split, a new root with a single key is created. The tree therefore grows only at the root, and the leaves stay on the same level.

Figure 2 repeats the insertion example of the course. The key 16 goes down from the root `[8 19 32]` through `child[1]` to the full leaf `[10 12 14 17]` and goes to position 4, between 14 and 17. The leaf

reaches 5 keys and is split into `[10 12]` and `[16 17]`, and the median 14 moves up to the root, which becomes `[8 14 19 32]` and still has room. In Figure 2 and the figures after it, the key just inserted has a green background and the keys moved by the operation have a yellow background; in a deletion, the deleted key is struck through.

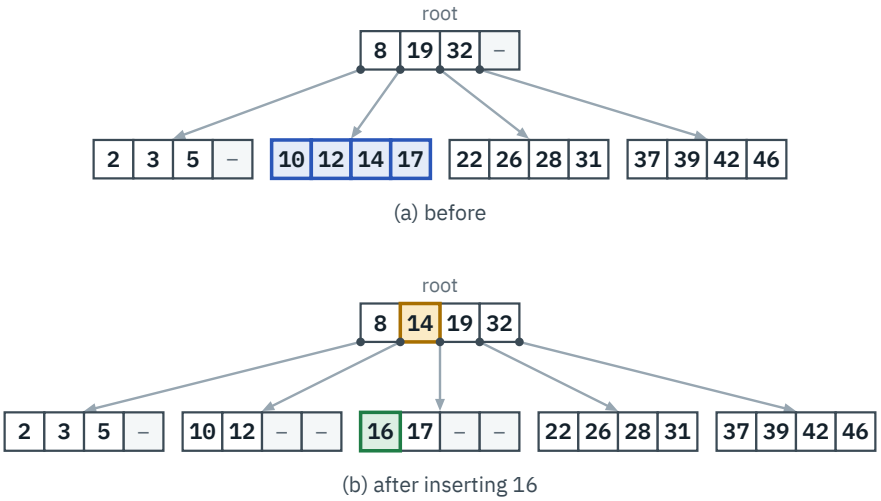


Figure 2. Inserting the key 16: (a) the leaf `[10 12 14 17]`, with a blue background, is full; (b) after receiving 16 it holds 5 keys, so it is split into `[10 12]` and `[16 17]`, and the median 14 moves up to the root. The tree of (a) is built by inserting, in this order, the keys `2, 3, 8, 10, 12, 5, 19, 22, 26, 32, 37, 39, 14, 17, 28, 31, 42, 46`.

The median can also move up into an internal node other than the root. In Figure 1, the leaf `[37 39 42 46]` is full. When 44 is inserted, the search goes down from the root through `child[1]` and from node `[26 32]` through `child[2]`. The leaf becomes `[37 39 42 44 46]` and is split into `[37 39]` and `[44 46]`, and the median 42 moves up to node `[26 32]`. That node becomes `[26 32 42]` and still has room, so the root does not change (Figure 3).

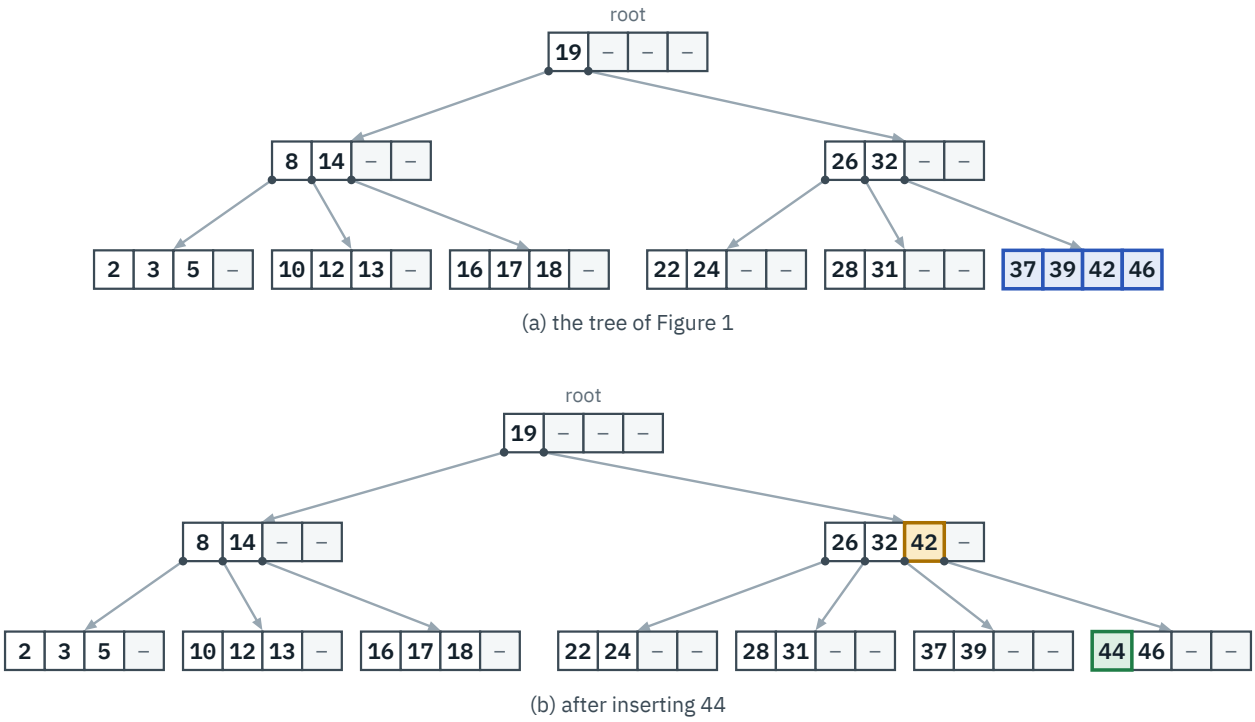


Figure 3. Inserting 44 into the tree of Figure 1: (a) the leaf `[37 39 42 46]`, with a blue background, is full; (b) after it is split, the median 42 has moved up to node `[26 32]`, which has become `[26 32 42]`.

If the parent is full too, the splits go on upwards. In the tree of Figure 2b, the root `[8 14 19 32]` already holds 4 keys. When 29 is inserted, the leaf `[22 26 28 31]` reaches 5 keys and is split into `[22 26]` and `[29 31]`, and the median 28 moves up to the root. The root reaches 5 keys as well, `[8 14 19 28 32]`, and is split in turn: `[8 14]` and `[28 32]` become the children of a new root, `[19]`, and the tree grows by one level (Figure 4). When the leaf has room, no node is split: for example, the key 11 goes into the leaf `[10 12]` of Figure 2b, which becomes `[10 11 12]`.

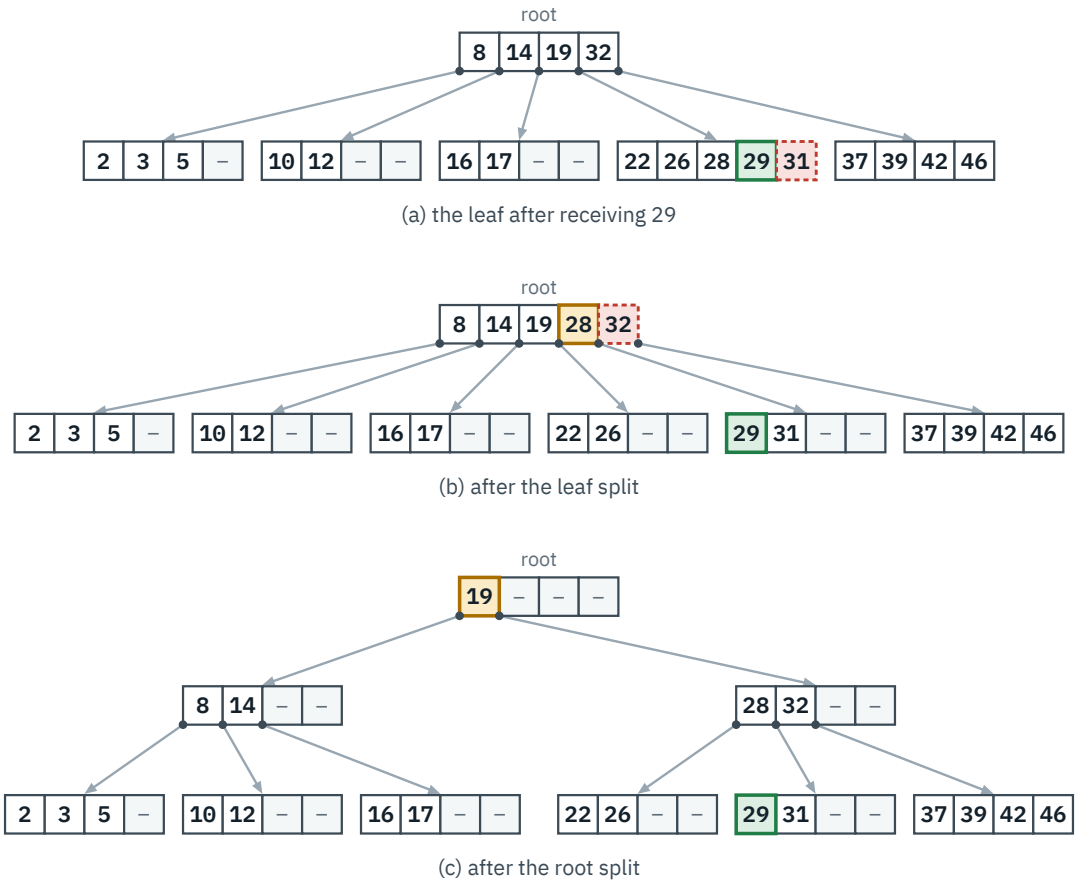


Figure 4. Inserting 29 into the tree of Figure 2b: (a) the leaf has received 29 and holds 5 keys; the dotted cell is the extra key; (b) the leaf has been split, and the median 28 has moved up to the root, which now holds 5 keys; (c) the root has been split, and the median 19 has moved up into a new root.

The B-TREE-INSERT pseudocode in the course, taken from [5], splits full nodes on the way down, in a single pass. The course examples and the program of this lab split a node only after it has received the extra key, on the way back up, as in [1, 4]. Both variants build correct B-trees, but not necessarily the same trees.

3.3 Deletion

If `k` is in a leaf, it is removed directly. If it is in an internal node, it is replaced with its predecessor, the largest key of the left subtree `child[i - 1]`. The predecessor is found by going down from `child[i - 1]`, always through the last child, to a leaf: it is the last key of that leaf. The predecessor is then deleted from the leaf. In Figure 1, the key 3 is deleted directly from the leaf `[2 3 5]`, and the key 19, in the root, is replaced with its predecessor 18, the last key of the leaf `[16 17 18]`. Both leaves keep 2 keys, so no fix is needed.

A deletion can leave a leaf with d − 1 keys. On the way back up, every child left with too few keys is fixed, in this order:

1. **Borrow from the right sibling**, if it holds more than d keys: the separating key in the parent moves down to the end of the child, and the sibling's first key moves up to take its place (Figure 5).
2. **Borrow from the left sibling**, if it holds more than d keys, with the mirrored operation: the separator moves down to the start of the child, and the sibling's last key moves up to take its

place (Figures 6 and 7).

3. **Merge**, if the siblings hold only d keys each: the child, the separating key and one sibling, the right one if there is one, become a single node with (d − 1) + 1 + d = 2d keys. The parent loses a key and may be left with too few keys in turn (Figures 8 and 9).

If the root is left without keys, its only child becomes the root, and the tree loses one level.

A borrow moves one key from a sibling on the same level, through the parent: the sibling's key cannot go straight into the child, since that would break the order around the separator, so it moves up in place of the separator, and the separator moves down into the child. In the tree of Figure 2b, deleting 17 leaves the leaf `[16]` with a single key. The right sibling `[22 26 28 31]` holds 4 keys, so it lends one: the separator 19 moves down from the root to the end of the leaf, which becomes `[16 19]`, and 22, the sibling's first key, moves up to take its place. The parent keeps the same number of keys, so the fix stops here (Figure 5).

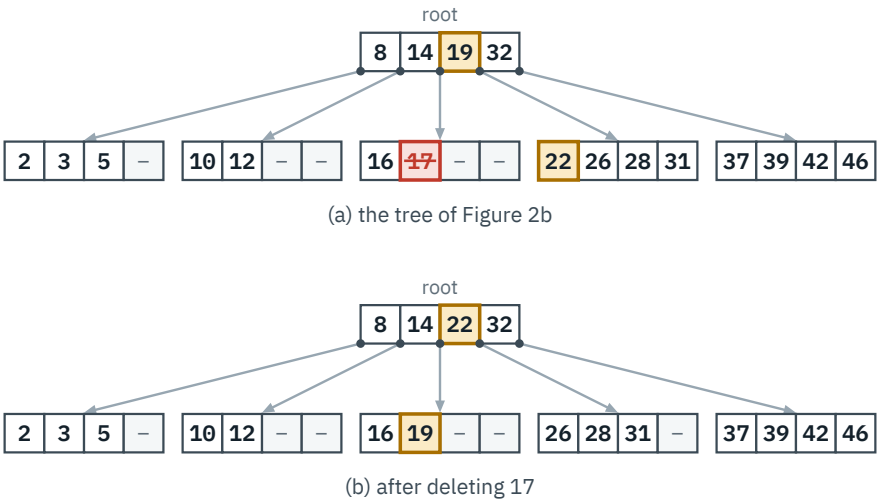


Figure 5. Borrowing from the right sibling, when 17 is deleted from the tree of Figure 2b: the leaf `[16]` receives the separator 19, and 22 moves up from the right sibling to take its place.

The right sibling is tried first. In the same tree of Figure 2b, deleting 12 leaves the leaf `[10]`. The right sibling `[16 17]` holds only 2 = d keys and cannot lend any, but the left sibling `[2 3 5]` holds 3: the separator 8 moves down to the start of the leaf, which becomes `[8 10]`, and 5, the last key of the left sibling, moves up to take its place (Figure 6). The same borrow happens when the deleted key is in an internal node: the predecessor takes its place, and the leaf it came from may be left below the minimum (Figure 7).

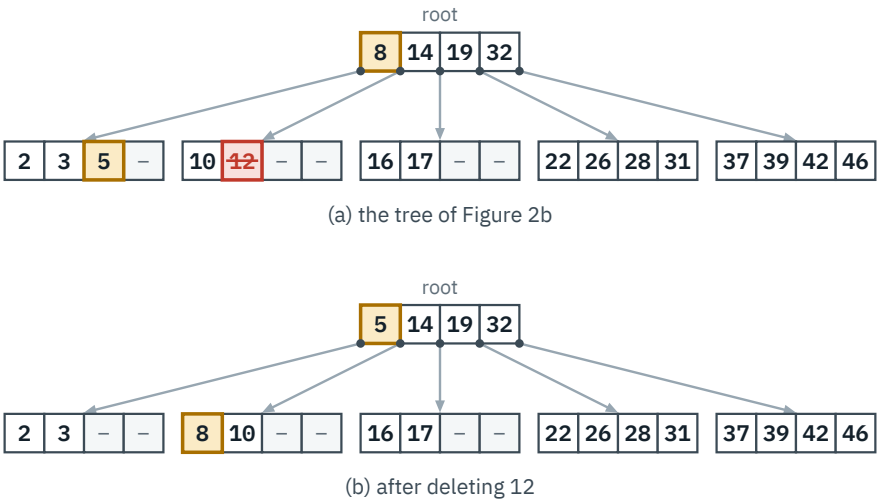


Figure 6. Borrowing from the left sibling, when 12 is deleted from the tree of Figure 2b: the right sibling `[16 17]` holds only 2 keys, so the leaf `[10]` receives the separator 8, and 5 moves up from the left sibling to take its place.

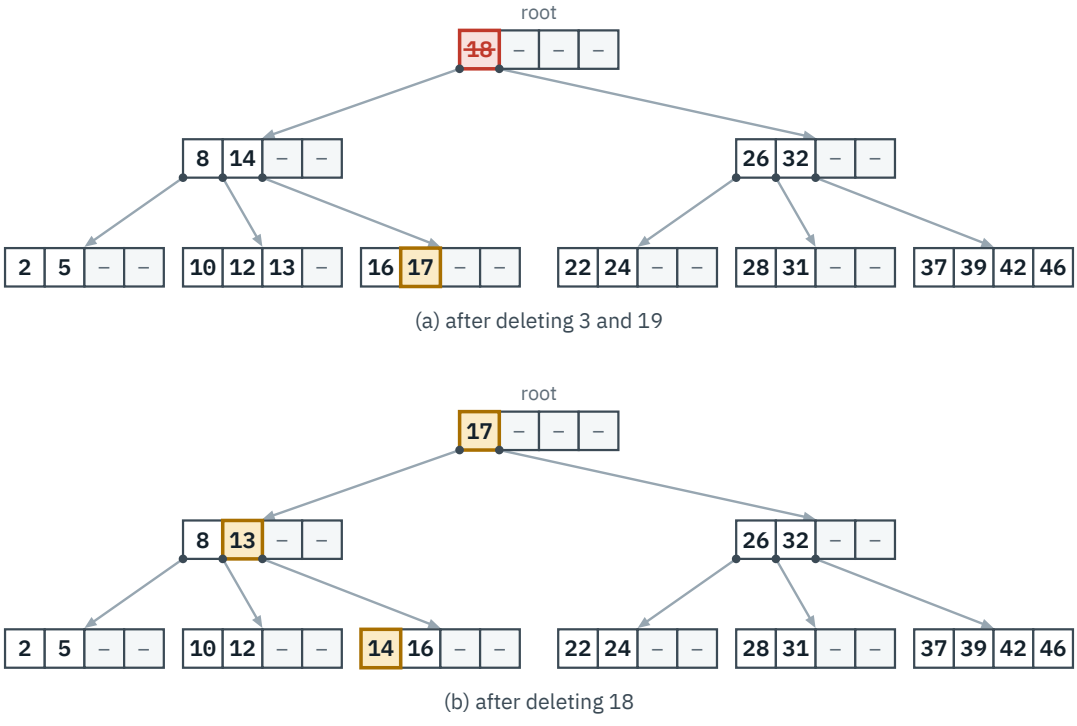
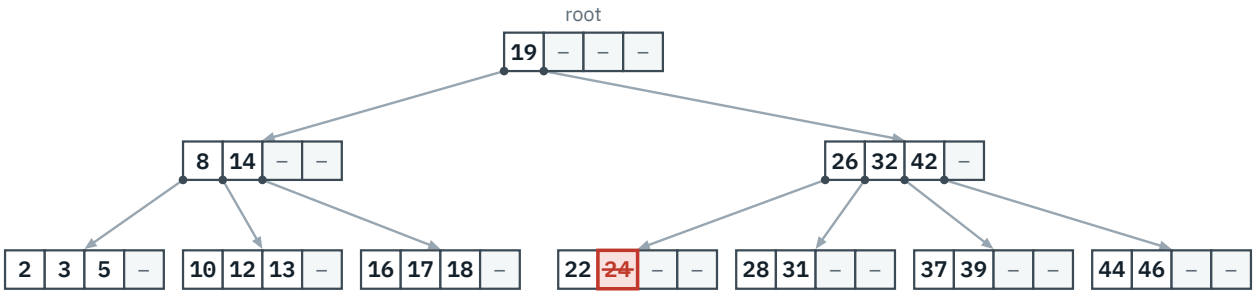
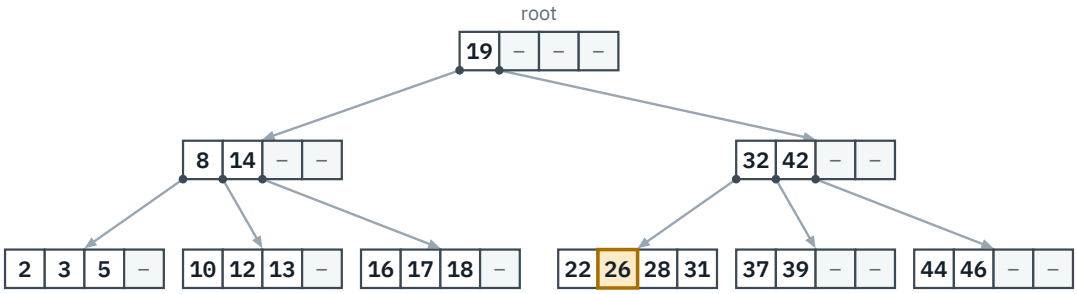


Figure 7. Deleting the key 18 from the root, after deleting 3 and 19, as in the course example: the predecessor 17 takes its place, and the leaf `[16]` is left with a single key. It has no right sibling, and the left sibling `[10 12 13]` holds 3 keys, so it lends one: the separator 14 moves down to the leaf, and 13 moves up to the parent.

A merge goes on towards the root only if the parent is left below the minimum. In the tree of Figure 3b, deleting 24 leaves the leaf `[22]`. It has no left sibling, and the right sibling `[28 31]` holds only 2 keys, so it cannot lend one: the leaf, the separator 26 and the right sibling become a single node, `[22]` + 26 + `[28 31]` = `[22 26 28 31]`. The parent loses the separator and becomes `[32 42]`, with 2 = d keys, so nothing else needs fixing (Figure 8). In Figure 9 the parent is left below the minimum, and the merges go on up to the root.

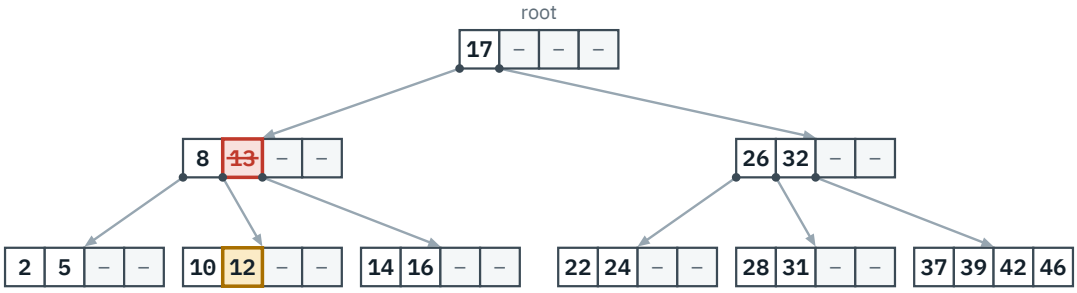


(a) the tree of Figure 3b

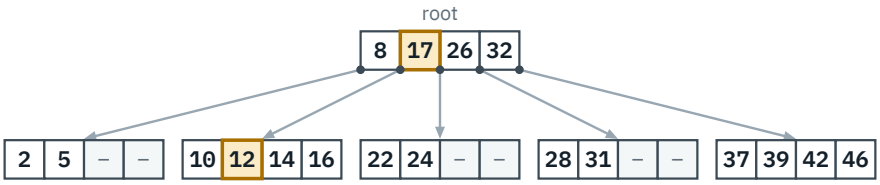


(b) after deleting 24

Figure 8. Deleting 24 from the tree of Figure 3b: the leaf `[22]` is merged with the separator 26 and the right sibling `[28 31]`, and the parent keeps 2 keys, `[32 42]`.



(a) the tree of Figure 7b



(b) after deleting 13

Figure 9. Deleting the key 13 from the tree of Figure 7b: the predecessor 12 takes its place, and the leaf `[10]` is merged with its right sibling, `[14 16]`. The node `[8]` is left with a single key and is merged with its right sibling, `[26 32]`. The root is left empty, and the tree loses one level.

3.4 Listing

An inorder traversal produces the keys in increasing order, with no sorting: in every node, before each key `key[i]` the subtree `child[i - 1]` is walked, and at the end the last child. Listing the keys of an interval [a, b] goes down only into the subtrees that can hold keys of the interval: the keys smaller than a are skipped, together with the subtrees on their left, and the walk of a node stops at the first key larger than b. On the tree of Figure 1, the interval [10, 30] holds 12 keys, found by visiting 7 of the 9 nodes; the leaves `[2 3 5]` and `[37 39 42 46]` are not visited.

3.5 Cost of the operations

Table 2 sums up the cost of the operations. The height is logarithmic, $h \le \log_{d+1}((n + 1) / 2)$, so $h = O(\log_d n)$. The "nodes visited" column counts the nodes read, that is, the number of disk accesses when the tree is stored on disk: $O(h) = O(\log_d n)$. The "any order" column adds the work inside each node: the program searches a node linearly, with at most 2d comparisons, and moves up to 2d keys in a split or a merge. For a fixed order, d is a constant, and every single-key operation costs O(log n), as in balanced binary trees, but the tree has several times fewer levels.

Table 2. Cost of the operations: *n* is the number of keys, *d* the order of the tree, *h* the height and *r* the number of keys listed.

OPERATION	TIME, FIXED ORDER	TIME, ANY ORDER	NODES VISITED	REASON
search	`O(log n)`	$O(d \cdot h) = O(d \cdot \log_d n)$	`h + 1`	one node per level, with at most 2d comparisons in each
insertion	`O(log n)`	$O(d \cdot h) = O(d \cdot \log_d n)$	`O(h)`	the descent, plus at most h + 1 splits of O(d) moves each
deletion	`O(log n)`	$O(d \cdot h) = O(d \cdot \log_d n)$	`O(h)`	the descent, the search for the predecessor and at most h fixes of O(d) moves each
listing in order	`O(n)`	`O(n)`	`all`	every node and every key is visited once
listing an interval	`O(log n + r)`	`O(d · h + r)`	`O(h + r / d)`	only the subtrees that can hold keys of the interval are visited

With a binary search inside the node, a search would cost O(log n) for any d; insertion and deletion stay $O(d \cdot \log_d n)$, because of the key moves. As for memory, a node takes 80 bytes for d = 2, and every node except the root is at least half full, so the tree takes O(n) memory.

4 Lab work

The functions in section 4.1 implement the operations of section 3, and the complete menu-driven program is in section 8. The lab goes as follows:

1. Study the `NodeB` structure and the functions of the `BTree` class in section 4.1. For every function, follow on the tree of Figure 1 the nodes it visits and the keys it moves.
2. Build the program from section 8 and create the tree of Figure 1 with option 2 of the menu, from the list of keys in the caption of the figure. Compare the level-by-level printout, option 5, with Figure 1.
3. Insert 44 into the tree of Figure 1, then delete 24, and compare the trees with Figures 3b and 8b. Empty the tree with option 9, build the tree of Figure 2a from the list of keys in the caption of the

figure, insert 16 and 29 and compare with Figures 2b and 4c.

4. Empty the tree and build the tree of Figure 2b, from the list of keys in the caption of Figure 2 followed by 16. Delete 17 and compare with Figure 5b. Build it again, delete 12 and compare with Figure 6b. Explain why, when 12 is deleted, the leaf does not borrow a key from its right sibling.
5. Build the tree of Figure 1 again and apply the operations of Table 3, in order. After every insertion and deletion, print the tree, check its properties with option 8 and compare the trees with Figures 7 and 9.
6. Solve the exercises in section 5 and answer the questions in sections 6 and 7.

4.1 The complete functions

The order constants, the `NodeB` structure and the `BTree` class, with all the operations of section 3. The function `position` gives the first position `i` with `k ≤ key[i]` and is used by search, insertion and deletion. The lines are numbered as in the complete program of section 8.

const int D · struct NodeB · class BTree

```
21  const int D = 2;          // the order of the tree
22  const int MAXK = 2 * D;   // the largest number of keys in a node
23  const int MINK = D;       // the smallest number of keys in a node other than the root
24
25  struct NodeB {
26      int nr;                   // the number of keys in the node
27      int key[MAXK + 2];        // key[1..nr]; key[MAXK + 1] briefly holds one extra key
28      NodeB* child[MAXK + 2];   // child[0..nr]; child[i]: keys from key[i] to key[i+1]
29
30      NodeB() : nr(0) {
31          for (int i = 0; i < MAXK + 2; i++) { key[i] = 0; child[i] = nullptr; }
32      }
33      bool leaf() const { return child[0] == nullptr; }
34  };
35
36  class BTree {
37  public:
38      BTree() : root(nullptr), count(0) {}
39      ~BTree() { clear(); }
40      BTree(const BTree&) = delete;
41      BTree& operator=(const BTree&) = delete;
42
43      int size() const { return count; }
44      bool empty() const { return root == nullptr; }
45
46      // searches for k; if path is not null, it receives the visited nodes, root first
47      bool search(int k, std::vector<const NodeB*>* path = nullptr) const {
48          const NodeB* n = root;
49          while (n != nullptr) {
50              if (path != nullptr) path->push_back(n);
51              int i = position(n, k);
52              if (i <= n->nr && n->key[i] == k) return true;
53              n = n->child[i - 1];              // at a leaf, child[i - 1] is nullptr
54          }
55          return false;
56      }
57
58      // inserts k; returns false if the key is already there
59      bool insert(int k) {
60          if (root == nullptr) {
61              root = new NodeB();
62              root->nr = 1;
63              root->key[1] = k;
64              count = 1;
65              return true;
66          }
67          int up = 0;
68          NodeB* right = nullptr;
69          Result r = insertIn(root, k, up, right);
70          if (r == EXISTS) return false;
```

```
        if (r == SPLIT) {                       // the root was split: one more level
            NodeB* s = new NodeB();
            s->nr = 1;
            s->key[1] = up;
            s->child[0] = root;
            s->child[1] = right;
            root = s;
        }
        count++;
        return true;
    }

    // deletes k; returns false if the key is not there
    bool remove(int k) {
        if (root == nullptr || !removeFrom(root, k)) return false;
        count--;
        if (root->nr == 0) {                    // the root has no keys left
            NodeB* old = root;
            root = root->child[0];              // nullptr if the tree is now empty
            delete old;
        }
        return true;
    }

    // all the keys, in increasing order
    std::vector<int> keys() const {
        std::vector<int> out;
        inorder(root, out);
        return out;
    }

    // the keys in the interval [a, b], in increasing order
    std::vector<int> range(int a, int b) const {
        std::vector<int> out;
        inRange(root, a, b, out);
        return out;
    }

    int nodes() const { return countNodes(root); }

    // the height: the number of edges from the root to the leaves (-1 if empty)
    int height() const {
        int h = -1;
        for (const NodeB* n = root; n != nullptr; n = n->child[0]) h++;
        return h;
    }

    // checks the B-tree properties; why receives the first property broken
    bool check(std::string& why) const {
        if (root == nullptr) {
            if (count == 0) return true;
            why = "the tree is empty, but the key counter is not 0";
            return false;
        }
        int leafLevel = -1, n = 0;
        if (!checkNode(root, 0, true, false, 0, false, 0, leafLevel, n, why))
            return false;
        if (n != count) {
            why = "the number of keys does not match the counter";
            return false;
        }
        return true;
    }

    // prints the tree level by level; "|" separates children of different parents
    void print() const {
        if (root == nullptr) { std::cout << "  (the tree is empty)\n"; return; }
        typedef std::vector<const NodeB*> Group;   // the children of one parent
        std::vector<Group> level(1, Group(1, root));
        for (int d = 0; !level.empty(); d++) {
            std::vector<Group> next;
            std::cout << "  level " << d << ":";
            for (size_t g = 0; g < level.size(); g++) {
                if (g > 0) std::cout << " |";
                for (const NodeB* n : level[g]) {
                    std::cout << ' ' << str(n);
```

```
                    if (n->leaf()) continue;
                    next.push_back(Group());
                    for (int i = 0; i <= n->nr; i++) next.back().push_back(n->child[i]);
                }
            }
            std::cout << '\n';
            level.swap(next);
        }
    }

    void clear() {
        destroy(root);
        root = nullptr;
        count = 0;
    }

    // a node written as in the course, for example [10 12 13]
    static std::string str(const NodeB* n) {
        std::string s = "[";
        for (int i = 1; i <= n->nr; i++) {
            if (i > 1) s += ' ';
            s += std::to_string(n->key[i]);
        }
        return s + "]";
    }

private:
    NodeB* root;
    int count;

    enum Result { EXISTS, DONE, SPLIT };

    // the first position i with k <= key[i], or nr + 1 if k exceeds every key
    static int position(const NodeB* n, int k) {
        int i = 1;
        while (i <= n->nr && k > n->key[i]) i++;
        return i;
    }

    // inserts k into the subtree rooted at n; if n is split, returns SPLIT,
    // with the median key in up and the new right node in right
    static Result insertIn(NodeB* n, int k, int& up, NodeB*& right) {
        int i = position(n, k);
        if (i <= n->nr && n->key[i] == k) return EXISTS;
        int x = k;                              // the key that goes into n
        NodeB* xr = nullptr;                    // its right child (null at leaves)
        if (!n->leaf()) {
            Result r = insertIn(n->child[i - 1], k, x, xr);
            if (r != SPLIT) return r;           // the child took the key without a split
        }
        for (int j = n->nr; j >= i; j--) {      // x goes to position i, xr to its right
            n->key[j + 1] = n->key[j];
            n->child[j + 1] = n->child[j];
        }
        n->key[i] = x;
        n->child[i] = xr;
        n->nr++;
        if (n->nr <= MAXK) return DONE;
        split(n, up, right);                    // n holds 2d + 1 keys: split it
        return SPLIT;
    }

    // n holds 2d + 1 keys: the first d stay in n, key d + 1 moves up to the parent,
    // and the last d move to the new node right
    static void split(NodeB* n, int& up, NodeB*& right) {
        right = new NodeB();
        right->child[0] = n->child[D + 1];
        n->child[D + 1] = nullptr;
        for (int j = 1; j <= D; j++) {
            right->key[j] = n->key[D + 1 + j];
            right->child[j] = n->child[D + 1 + j];
            n->child[D + 1 + j] = nullptr;
        }
        right->nr = D;
        up = n->key[D + 1];
        n->nr = D;
```

```
    }

    // deletes k from the subtree rooted at n; a child left with d - 1 keys is fixed
    static bool removeFrom(NodeB* n, int k) {
        int i = position(n, k);
        bool here = i <= n->nr && n->key[i] == k;
        if (n->leaf()) {
            if (!here) return false;
            for (int j = i; j < n->nr; j++) n->key[j] = n->key[j + 1];
            n->nr--;
            return true;
        }
        if (here) {
            // internal node: k is replaced with its predecessor, the largest key of
            // the left subtree; the predecessor is then deleted from its leaf
            const NodeB* p = n->child[i - 1];
            while (!p->leaf()) p = p->child[p->nr];
            n->key[i] = p->key[p->nr];
            k = n->key[i];
        }
        if (!removeFrom(n->child[i - 1], k)) return false;
        if (n->child[i - 1]->nr < MINK) fix(n, i - 1);
        return true;
    }

    // child c of n holds d - 1 keys: first try to borrow from the right sibling,
    // then from the left one; if both hold only d keys, the child is merged
    static void fix(NodeB* n, int c) {
        if (c < n->nr && n->child[c + 1]->nr > MINK) borrowRight(n, c);
        else if (c > 0 && n->child[c - 1]->nr > MINK) borrowLeft(n, c);
        else if (c < n->nr) merge(n, c);       // with the right sibling
        else merge(n, c - 1);                  // with the left sibling
    }

    // key[c + 1] moves down to the end of child c; the right sibling's first key
    // moves up
    static void borrowRight(NodeB* n, int c) {
        NodeB* a = n->child[c];
        NodeB* b = n->child[c + 1];
        a->nr++;
        a->key[a->nr] = n->key[c + 1];
        a->child[a->nr] = b->child[0];
        n->key[c + 1] = b->key[1];
        b->child[0] = b->child[1];
        for (int j = 1; j < b->nr; j++) {
            b->key[j] = b->key[j + 1];
            b->child[j] = b->child[j + 1];
        }
        b->child[b->nr] = nullptr;
        b->nr--;
    }

    // key[c] moves down to the front of child c; the left sibling's last key moves up
    static void borrowLeft(NodeB* n, int c) {
        NodeB* a = n->child[c - 1];
        NodeB* b = n->child[c];
        for (int j = b->nr; j >= 1; j--) {
            b->key[j + 1] = b->key[j];
            b->child[j + 1] = b->child[j];
        }
        b->child[1] = b->child[0];
        b->key[1] = n->key[c];
        b->child[0] = a->child[a->nr];
        b->nr++;
        n->key[c] = a->key[a->nr];
        a->child[a->nr] = nullptr;
        a->nr--;
    }

    // children c and c + 1 and the key key[c + 1] between them become one node
    static void merge(NodeB* n, int c) {
        NodeB* a = n->child[c];
        NodeB* b = n->child[c + 1];
        a->key[a->nr + 1] = n->key[c + 1];
        a->child[a->nr + 1] = b->child[0];
        for (int j = 1; j <= b->nr; j++) {     // the keys and children of b
```

```
            a->key[a->nr + 1 + j] = b->key[j];
            a->child[a->nr + 1 + j] = b->child[j];
        }
        a->nr += 1 + b->nr;
        for (int j = c + 1; j < n->nr; j++) {  // key[c + 1] and child[c + 1] leave n
            n->key[j] = n->key[j + 1];
            n->child[j] = n->child[j + 1];
        }
        n->child[n->nr] = nullptr;
        n->nr--;
        delete b;
    }

    static void inorder(const NodeB* n, std::vector<int>& out) {
        if (n == nullptr) return;
        for (int i = 1; i <= n->nr; i++) {
            inorder(n->child[i - 1], out);
            out.push_back(n->key[i]);
        }
        inorder(n->child[n->nr], out);
    }

    // goes down only into children that may hold keys in [a, b]
    static void inRange(const NodeB* n, int a, int b, std::vector<int>& out) {
        if (n == nullptr) return;
        int i = position(n, a);                // key[1..i-1] are smaller than a
        for (; i <= n->nr && n->key[i] <= b; i++) {
            inRange(n->child[i - 1], a, b, out);
            out.push_back(n->key[i]);
        }
        inRange(n->child[i - 1], a, b, out);   // the child after the last key <= b
    }

    static int countNodes(const NodeB* n) {
        if (n == nullptr) return 0;
        int c = 1;
        if (!n->leaf())
            for (int i = 0; i <= n->nr; i++) c += countNodes(n->child[i]);
        return c;
    }

    // the keys of n must lie in the interval (lo, hi); hasLo/hasHi say whether
    // the interval has a left and a right end
    static bool checkNode(const NodeB* n, int depth, bool isRoot, bool hasLo, int lo,
                          bool hasHi, int hi, int& leafLevel, int& total,
                          std::string& why) {
        if (n->nr < (isRoot ? 1 : MINK) || n->nr > MAXK) {
            why = "node " + str(n) + " holds " + std::to_string(n->nr) + " keys";
            return false;
        }
        for (int i = 1; i <= n->nr; i++) {
            if ((i > 1 && n->key[i] <= n->key[i - 1]) || (hasLo && n->key[i] <= lo) ||
                (hasHi && n->key[i] >= hi)) {
                why = "the keys of node " + str(n) + " are out of order";
                return false;
            }
        }
        total += n->nr;
        if (n->leaf()) {
            for (int i = 1; i <= n->nr; i++)
                if (n->child[i] != nullptr) {
                    why = "leaf " + str(n) + " has a child";
                    return false;
                }
            if (leafLevel < 0) leafLevel = depth;
            if (depth != leafLevel) {
                why = "the leaves are not all on the same level";
                return false;
            }
            return true;
        }
        for (int i = 0; i <= n->nr; i++) {
            if (n->child[i] == nullptr) {
                why = "internal node " + str(n) + " has a null child";
                return false;
            }
```

```
            bool l = i > 0 || hasLo, h = i < n->nr || hasHi;
            int a = i > 0 ? n->key[i] : lo, b = i < n->nr ? n->key[i + 1] : hi;
            if (!checkNode(n->child[i], depth + 1, false, l, a, h, b, leafLevel, total,
                           why))
                return false;
        }
        return true;
    }

    static void destroy(NodeB* n) {
        if (n == nullptr) return;
        for (int i = 0; i <= n->nr; i++) destroy(n->child[i]);
        delete n;
    }
};
```

4.2 Expected results

Table 3 lists the results of the operations applied, in order, to the tree of Figure 1: every operation starts from the tree left by the one before. The column "h · nodes" gives the height and the number of nodes after the operation; the program prints them in the menu header.

Table 3. Results of the operations applied, in order, to the tree of Figure 1.

OPERATION	RESULT	H · NODES	EXPLANATION
`t.search(24)`	`true`	2 · 9	path `[19]` → `[26 32]` → `[22 24]`; 3 nodes visited
`t.search(11)`	`false`	2 · 9	path `[19]` → `[8 14]` → `[10 12 13]`; the leaf does not hold 11
`t.range(10, 30)`	`10 12 13 14 16 17 18 19 22 24 26 28`	2 · 9	12 keys; 7 of the 9 nodes visited
`t.remove(3)`	`true`	2 · 9	from a leaf, with no fix
`t.remove(19)`	`true`	2 · 9	replaced with its predecessor 18, with no fix
`t.remove(18)`	`true`	2 · 9	replaced with its predecessor 17; borrowing from the left sibling
`t.remove(13)`	`true`	1 · 6	replaced with its predecessor 12; merging with the right sibling, twice; the height drops to 1
`t.insert(40)`	`true`	2 · 9	2 splits, the root included; the height grows to 2
`t.insert(13)`	`true`	2 · 10	1 split
`t.remove(26)`	`true`	2 · 9	replaced with its predecessor 24; merging with the left sibling

5 Exercises

1. **Binary search in a node.** Replace the linear search in `position` with a binary search. Count the key comparisons when searching for 10^5 random keys, for d = 2 and for d = 50, with both variants.
2. **Deletion with the successor.** Change `removeFrom` so that a key in an internal node is replaced with its successor, the smallest key of `child[i]`, and make `fix` try the left sibling first, as in the code of the ADS repository. Apply both variants to the tree of Figure 1, with the deletions of Table 3, and compare the trees you get.

3. **Single-pass insertion.** Implement the B-TREE-INSERT pseudocode of the course, with the minimum degree t: every full node, with $2t - 1$ keys, met on the way down is split before going into it. For $t = 3$, compare the tree you get after inserting the keys 1…17 with the one the program of this lab builds with d = 2.
4. **Order statistics.** Keep in every node the total number of keys in its subtree and update it on insertion, split, borrowing and merging. Implement `int kth(int k)`, the k-th smallest key, and `int rank(int x)`, the number of keys smaller than x, both in $O(d \cdot h)$.
5. **B+ tree.** Keep every key in the leaves, and in the internal nodes only copies of keys, used as separators. Link the leaves in a list and implement interval listing by going down once and then walking the list of leaves.
6. **Building from sorted keys.** Build a B-tree from n sorted keys directly, bottom up, in O(n): fill the leaves from left to right, then the levels above, keeping at least d keys in every node. Compare the number of nodes with the tree you get by inserting the keys one by one.
7. **Bonus: nodes on disk.** Store every node in a binary file as a fixed-size page, with the page index in place of the pointer. Keep only the root in memory and count the pages read and written by search, insertion and deletion, for d = 2 and for d = 50, on 10^5 keys.

6 Practice questions

Answer the questions on paper, drawing the tree where it helps.

1. Insert the keys 1, 2, …, 17, in this order, into an empty B-tree with d = 2, the task on slide 7 of the course. Draw the tree after every split. How many splits happen, and what is the height of the final tree?
2. Delete from the tree of question 1, in this order, the keys 9, 16 and 1. Which fix is used for each deletion, and what is the height of the tree at the end?
3. On the tree of Figure 1, how many nodes does the search for 24, 11 and 46 visit? In which node does each search stop?
4. Insert the same keys 1…17 with d = 1, that is into a 2-3 tree. What height and how many nodes does the tree have? Compare with question 1.
5. What are the smallest and the largest number of keys in a B-tree with d = 2 and height 2? Draw an example of each.
6. Give two insertion orders of the keys 1…5 that build different trees with d = 1. Why does the shape of a B-tree depend on the order of the insertions?

7 Review questions

1. Why are all the leaves of a B-tree on the same level, after any sequence of insertions and deletions?
2. Why must every node except the root hold at least d keys? What would happen to the height without this rule?
3. Why does the median move up to the parent in a split, and not some other key?

4. Where is the predecessor of a key of an internal node always found? Why can deleting it not lead to another search for a predecessor?
5. When is a key borrowed and when are two nodes merged? Why does a merged node never exceed 2d keys?
6. How does the order m of the course, the largest number of children, relate to the order d of the code? What order m does a 2-3 tree have?
7. Why do B-trees suit data stored on disk? What is the largest height of a tree with 10^6 keys for d = 2 and for d = 50?

8 Source code

The complete menu-driven program starts from the implementation in the course's ADS repository [6] and keeps its node structure, which is also the one of the course: the number of keys `nr`, the keys `key[1..nr]` and the pointers `child[0..nr]`. The code is rewritten in C++. Compared with the original:

- the search goes down into the right child; the original always went down into the last child, so after inserting the keys 1…8 it no longer found the keys 1, 2, 4 and 5;
- merging with the left sibling copies the child pointers as well, not only the keys; in the original, after inserting the keys 1…17 and deleting the key 9, the leaf with the keys 16 and 17 was lost, and printing the tree crashed the program;
- deleting a missing key and deleting from an empty tree no longer crash the program;
- nodes are allocated with `new`, with every pointer initialized, and the destructor of the `BTree` class frees the memory;
- the insertion first puts the key into the node and only then splits it, as in the course, instead of the three cases of `placeInPage`; the deletion follows the course examples, with the predecessor and with the right sibling tried first for borrowing;
- printing by levels, interval listing, a property check and a menu that validates its input were added, and the program runs on any system, without `conio.h`.

Build: `g++ -std=c++11 -Wall -Wextra -o btree_menu_en btree_menu_en.cpp`. The order is set by the constant `D`.

btree_menu_en.cpp

```
// B-trees - DSA lab
//
// B-tree of order d (Bayer and McCreight, 1972), as in the examples of the course:
//  - every node holds at most 2d keys; an internal node with k keys has k + 1 children;
//  - every node except the root holds at least d keys;
//  - all leaves are on the same level;
//  - the keys of every node are in increasing order.
// With d = 2, a node holds 2 to 4 keys and has 3 to 5 children. In the definition of
// the course, where the order m is the maximum number of children, this is a B-tree
// of order m = 5.
//
// Build: g++ -std=c++11 -Wall -Wextra -o btree_menu_en btree_menu_en.cpp

#include <cerrno>
#include <climits>
#include <cstdlib>
#include <iostream>
#include <string>
#include <vector>
```

```
20
21  const int D = 2;          // the order of the tree
22  const int MAXK = 2 * D;   // the largest number of keys in a node
23  const int MINK = D;       // the smallest number of keys in a node other than the root
24
25  struct NodeB {
26      int nr;                   // the number of keys in the node
27      int key[MAXK + 2];        // key[1..nr]; key[MAXK + 1] briefly holds one extra key
28      NodeB* child[MAXK + 2];   // child[0..nr]; child[i]: keys from key[i] to key[i+1]
29
30      NodeB() : nr(0) {
31          for (int i = 0; i < MAXK + 2; i++) { key[i] = 0; child[i] = nullptr; }
32      }
33      bool leaf() const { return child[0] == nullptr; }
34  };
35
36  class BTree {
37  public:
38      BTree() : root(nullptr), count(0) {}
39      ~BTree() { clear(); }
40      BTree(const BTree&) = delete;
41      BTree& operator=(const BTree&) = delete;
42
43      int size() const { return count; }
44      bool empty() const { return root == nullptr; }
45
46      // searches for k; if path is not null, it receives the visited nodes, root first
47      bool search(int k, std::vector<const NodeB*>* path = nullptr) const {
48          const NodeB* n = root;
49          while (n != nullptr) {
50              if (path != nullptr) path->push_back(n);
51              int i = position(n, k);
52              if (i <= n->nr && n->key[i] == k) return true;
53              n = n->child[i - 1];              // at a leaf, child[i - 1] is nullptr
54          }
55          return false;
56      }
57
58      // inserts k; returns false if the key is already there
59      bool insert(int k) {
60          if (root == nullptr) {
61              root = new NodeB();
62              root->nr = 1;
63              root->key[1] = k;
64              count = 1;
65              return true;
66          }
67          int up = 0;
68          NodeB* right = nullptr;
69          Result r = insertIn(root, k, up, right);
70          if (r == EXISTS) return false;
71          if (r == SPLIT) {                     // the root was split: one more level
72              NodeB* s = new NodeB();
73              s->nr = 1;
74              s->key[1] = up;
75              s->child[0] = root;
76              s->child[1] = right;
77              root = s;
78          }
79          count++;
80          return true;
81      }
82
83      // deletes k; returns false if the key is not there
84      bool remove(int k) {
85          if (root == nullptr || !removeFrom(root, k)) return false;
86          count--;
87          if (root->nr == 0) {                  // the root has no keys left
88              NodeB* old = root;
89              root = root->child[0];            // nullptr if the tree is now empty
90              delete old;
91          }
92          return true;
93      }
94
95      // all the keys, in increasing order
```

```
    std::vector<int> keys() const {
        std::vector<int> out;
        inorder(root, out);
        return out;
    }

    // the keys in the interval [a, b], in increasing order
    std::vector<int> range(int a, int b) const {
        std::vector<int> out;
        inRange(root, a, b, out);
        return out;
    }

    int nodes() const { return countNodes(root); }

    // the height: the number of edges from the root to the leaves (-1 if empty)
    int height() const {
        int h = -1;
        for (const NodeB* n = root; n != nullptr; n = n->child[0]) h++;
        return h;
    }

    // checks the B-tree properties; why receives the first property broken
    bool check(std::string& why) const {
        if (root == nullptr) {
            if (count == 0) return true;
            why = "the tree is empty, but the key counter is not 0";
            return false;
        }
        int leafLevel = -1, n = 0;
        if (!checkNode(root, 0, true, false, 0, false, 0, leafLevel, n, why))
            return false;
        if (n != count) {
            why = "the number of keys does not match the counter";
            return false;
        }
        return true;
    }

    // prints the tree level by level; "|" separates children of different parents
    void print() const {
        if (root == nullptr) { std::cout << "  (the tree is empty)\n"; return; }
        typedef std::vector<const NodeB*> Group;   // the children of one parent
        std::vector<Group> level(1, Group(1, root));
        for (int d = 0; !level.empty(); d++) {
            std::vector<Group> next;
            std::cout << "  level " << d << ":";
            for (size_t g = 0; g < level.size(); g++) {
                if (g > 0) std::cout << " |";
                for (const NodeB* n : level[g]) {
                    std::cout << ' ' << str(n);
                    if (n->leaf()) continue;
                    next.push_back(Group());
                    for (int i = 0; i <= n->nr; i++) next.back().push_back(n->child[i]);
                }
            }
            std::cout << '\n';
            level.swap(next);
        }
    }

    void clear() {
        destroy(root);
        root = nullptr;
        count = 0;
    }

    // a node written as in the course, for example [10 12 13]
    static std::string str(const NodeB* n) {
        std::string s = "[";
        for (int i = 1; i <= n->nr; i++) {
            if (i > 1) s += ' ';
            s += std::to_string(n->key[i]);
        }
        return s + "]";
    }
```

```
172
173 private:
174     NodeB* root;
175     int count;
176
177     enum Result { EXISTS, DONE, SPLIT };
178
179     // the first position i with k <= key[i], or nr + 1 if k exceeds every key
180     static int position(const NodeB* n, int k) {
181         int i = 1;
182         while (i <= n->nr && k > n->key[i]) i++;
183         return i;
184     }
185
186     // inserts k into the subtree rooted at n; if n is split, returns SPLIT,
187     // with the median key in up and the new right node in right
188     static Result insertIn(NodeB* n, int k, int& up, NodeB*& right) {
189         int i = position(n, k);
190         if (i <= n->nr && n->key[i] == k) return EXISTS;
191         int x = k;                              // the key that goes into n
192         NodeB* xr = nullptr;                    // its right child (null at leaves)
193         if (!n->leaf()) {
194             Result r = insertIn(n->child[i - 1], k, x, xr);
195             if (r != SPLIT) return r;           // the child took the key without a split
196         }
197         for (int j = n->nr; j >= i; j--) {      // x goes to position i, xr to its right
198             n->key[j + 1] = n->key[j];
199             n->child[j + 1] = n->child[j];
200         }
201         n->key[i] = x;
202         n->child[i] = xr;
203         n->nr++;
204         if (n->nr <= MAXK) return DONE;
205         split(n, up, right);                    // n holds 2d + 1 keys: split it
206         return SPLIT;
207     }
208
209     // n holds 2d + 1 keys: the first d stay in n, key d + 1 moves up to the parent,
210     // and the last d move to the new node right
211     static void split(NodeB* n, int& up, NodeB*& right) {
212         right = new NodeB();
213         right->child[0] = n->child[D + 1];
214         n->child[D + 1] = nullptr;
215         for (int j = 1; j <= D; j++) {
216             right->key[j] = n->key[D + 1 + j];
217             right->child[j] = n->child[D + 1 + j];
218             n->child[D + 1 + j] = nullptr;
219         }
220         right->nr = D;
221         up = n->key[D + 1];
222         n->nr = D;
223     }
224
225     // deletes k from the subtree rooted at n; a child left with d - 1 keys is fixed
226     static bool removeFrom(NodeB* n, int k) {
227         int i = position(n, k);
228         bool here = i <= n->nr && n->key[i] == k;
229         if (n->leaf()) {
230             if (!here) return false;
231             for (int j = i; j < n->nr; j++) n->key[j] = n->key[j + 1];
232             n->nr--;
233             return true;
234         }
235         if (here) {
236             // internal node: k is replaced with its predecessor, the largest key of
237             // the left subtree; the predecessor is then deleted from its leaf
238             const NodeB* p = n->child[i - 1];
239             while (!p->leaf()) p = p->child[p->nr];
240             n->key[i] = p->key[p->nr];
241             k = n->key[i];
242         }
243         if (!removeFrom(n->child[i - 1], k)) return false;
244         if (n->child[i - 1]->nr < MINK) fix(n, i - 1);
245         return true;
246     }
247
```

```
    // child c of n holds d - 1 keys: first try to borrow from the right sibling,
    // then from the left one; if both hold only d keys, the child is merged
    static void fix(NodeB* n, int c) {
        if (c < n->nr && n->child[c + 1]->nr > MINK) borrowRight(n, c);
        else if (c > 0 && n->child[c - 1]->nr > MINK) borrowLeft(n, c);
        else if (c < n->nr) merge(n, c);       // with the right sibling
        else merge(n, c - 1);                  // with the left sibling
    }

    // key[c + 1] moves down to the end of child c; the right sibling's first key
    // moves up
    static void borrowRight(NodeB* n, int c) {
        NodeB* a = n->child[c];
        NodeB* b = n->child[c + 1];
        a->nr++;
        a->key[a->nr] = n->key[c + 1];
        a->child[a->nr] = b->child[0];
        n->key[c + 1] = b->key[1];
        b->child[0] = b->child[1];
        for (int j = 1; j < b->nr; j++) {
            b->key[j] = b->key[j + 1];
            b->child[j] = b->child[j + 1];
        }
        b->child[b->nr] = nullptr;
        b->nr--;
    }

    // key[c] moves down to the front of child c; the left sibling's last key moves up
    static void borrowLeft(NodeB* n, int c) {
        NodeB* a = n->child[c - 1];
        NodeB* b = n->child[c];
        for (int j = b->nr; j >= 1; j--) {
            b->key[j + 1] = b->key[j];
            b->child[j + 1] = b->child[j];
        }
        b->child[1] = b->child[0];
        b->key[1] = n->key[c];
        b->child[0] = a->child[a->nr];
        b->nr++;
        n->key[c] = a->key[a->nr];
        a->child[a->nr] = nullptr;
        a->nr--;
    }

    // children c and c + 1 and the key key[c + 1] between them become one node
    static void merge(NodeB* n, int c) {
        NodeB* a = n->child[c];
        NodeB* b = n->child[c + 1];
        a->key[a->nr + 1] = n->key[c + 1];
        a->child[a->nr + 1] = b->child[0];
        for (int j = 1; j <= b->nr; j++) {     // the keys and children of b
            a->key[a->nr + 1 + j] = b->key[j];
            a->child[a->nr + 1 + j] = b->child[j];
        }
        a->nr += 1 + b->nr;
        for (int j = c + 1; j < n->nr; j++) {  // key[c + 1] and child[c + 1] leave n
            n->key[j] = n->key[j + 1];
            n->child[j] = n->child[j + 1];
        }
        n->child[n->nr] = nullptr;
        n->nr--;
        delete b;
    }

    static void inorder(const NodeB* n, std::vector<int>& out) {
        if (n == nullptr) return;
        for (int i = 1; i <= n->nr; i++) {
            inorder(n->child[i - 1], out);
            out.push_back(n->key[i]);
        }
        inorder(n->child[n->nr], out);
    }

    // goes down only into children that may hold keys in [a, b]
    static void inRange(const NodeB* n, int a, int b, std::vector<int>& out) {
        if (n == nullptr) return;
```

```
        int i = position(n, a);                // key[1..i-1] are smaller than a
        for (; i <= n->nr && n->key[i] <= b; i++) {
            inRange(n->child[i - 1], a, b, out);
            out.push_back(n->key[i]);
        }
        inRange(n->child[i - 1], a, b, out);   // the child after the last key <= b
    }

    static int countNodes(const NodeB* n) {
        if (n == nullptr) return 0;
        int c = 1;
        if (!n->leaf())
            for (int i = 0; i <= n->nr; i++) c += countNodes(n->child[i]);
        return c;
    }

    // the keys of n must lie in the interval (lo, hi); hasLo/hasHi say whether
    // the interval has a left and a right end
    static bool checkNode(const NodeB* n, int depth, bool isRoot, bool hasLo, int lo,
                          bool hasHi, int hi, int& leafLevel, int& total,
                          std::string& why) {
        if (n->nr < (isRoot ? 1 : MINK) || n->nr > MAXK) {
            why = "node " + str(n) + " holds " + std::to_string(n->nr) + " keys";
            return false;
        }
        for (int i = 1; i <= n->nr; i++) {
            if ((i > 1 && n->key[i] <= n->key[i - 1]) || (hasLo && n->key[i] <= lo) ||
                (hasHi && n->key[i] >= hi)) {
                why = "the keys of node " + str(n) + " are out of order";
                return false;
            }
        }
        total += n->nr;
        if (n->leaf()) {
            for (int i = 1; i <= n->nr; i++)
                if (n->child[i] != nullptr) {
                    why = "leaf " + str(n) + " has a child";
                    return false;
                }
            if (leafLevel < 0) leafLevel = depth;
            if (depth != leafLevel) {
                why = "the leaves are not all on the same level";
                return false;
            }
            return true;
        }
        for (int i = 0; i <= n->nr; i++) {
            if (n->child[i] == nullptr) {
                why = "internal node " + str(n) + " has a null child";
                return false;
            }
            bool l = i > 0 || hasLo, h = i < n->nr || hasHi;
            int a = i > 0 ? n->key[i] : lo, b = i < n->nr ? n->key[i + 1] : hi;
            if (!checkNode(n->child[i], depth + 1, false, l, a, h, b, leafLevel, total,
                           why))
                return false;
        }
        return true;
    }

    static void destroy(NodeB* n) {
        if (n == nullptr) return;
        for (int i = 0; i <= n->nr; i++) destroy(n->child[i]);
        delete n;
    }
};

// ---------------------------------------------------------------- menu

// reads a line; returns false at the end of the input
static bool readLine(const std::string& prompt, std::string& line) {
    std::cout << prompt;
    return static_cast<bool>(std::getline(std::cin, line));
}

static bool isSep(char c) {
```

```
    return c == ' ' || c == '\t' || c == ',' || c == ';' || c == '\r';
}

// reads the integers on a line; separators: spaces, commas, semicolons
static bool parseInts(const std::string& line, std::vector<int>& out) {
    out.clear();
    size_t i = 0;
    while (i < line.size()) {
        if (isSep(line[i])) { i++; continue; }
        size_t j = i;
        while (j < line.size() && !isSep(line[j])) j++;
        std::string tok = line.substr(i, j - i);
        errno = 0;
        char* end = nullptr;
        long v = std::strtol(tok.c_str(), &end, 10);
        if (*end != '\0' || errno == ERANGE || v < INT_MIN || v > INT_MAX) return false;
        out.push_back(static_cast<int>(v));
        i = j;
    }
    return true;
}

// reads exactly one integer
static bool readInt(const std::string& prompt, int& k, bool& eof) {
    std::string line;
    eof = !readLine(prompt, line);
    if (eof) return false;
    std::vector<int> v;
    if (!parseInts(line, v) || v.size() != 1) {
        std::cout << "Invalid value: enter a single integer.\n";
        return false;
    }
    k = v[0];
    return true;
}

// "1 key", "5 keys"
static std::string plural(int n, const char* one, const char* many) {
    return std::to_string(n) + " " + (n == 1 ? one : many);
}

static void printList(const std::vector<int>& v) {
    if (v.empty()) { std::cout << "  (no keys)\n"; return; }
    std::cout << " ";
    for (int k : v) std::cout << ' ' << k;
    std::cout << "\n  (" << plural(static_cast<int>(v.size()), "key", "keys")
              << ")\n";
}

int main() {
    BTree t;
    std::string line;
    std::cout << "B-trees - DSA lab\n"
              << "Order d = " << D << ": every node holds at most " << MAXK
              << " keys, and every node except the root holds at least " << MINK
              << ".\n";
    for (;;) {
        std::cout << "\n=== B-tree | " << plural(t.size(), "key", "keys") << " | "
                  << plural(t.nodes(), "node", "nodes") << " | height " << t.height()
                  << " ===\n"
                  << " 1. Insert a key\n"
                  << " 2. Insert several keys (on one line)\n"
                  << " 3. Search for a key\n"
                  << " 4. Delete a key\n"
                  << " 5. Print the tree level by level\n"
                  << " 6. Print the keys in increasing order\n"
                  << " 7. Print the keys in an interval [a, b]\n"
                  << " 8. Check the B-tree properties\n"
                  << " 9. Empty the tree\n"
                  << " 0. Exit\n";
        if (!readLine("Option: ", line)) break;
        std::vector<int> opt;
        if (!parseInts(line, opt) || opt.size() != 1 || opt[0] < 0 || opt[0] > 9) {
            std::cout << "Invalid option.\n";
            continue;
        }
```

```
476         int o = opt[0];
477         bool eof = false;
478         int k = 0;
479         if (o == 0) break;
480         if (o == 1 || o == 3 || o == 4) {
481             if (!readInt("Key: ", k, eof)) { if (eof) break; continue; }
482             if (o == 1) {
483                 if (t.insert(k)) {
484                     std::cout << "Inserted key " << k << ".\n";
485                     t.print();
486                 } else {
487                     std::cout << "Key " << k << " is already in the tree.\n";
488                 }
489             } else if (o == 3) {
490                 std::vector<const NodeB*> path;
491                 bool found = t.search(k, &path);
492                 std::cout << "  path:";
493                 for (size_t i = 0; i < path.size(); i++)
494                     std::cout << (i ? " -> " : " ") << BTree::str(path[i]);
495                 if (path.empty()) std::cout << " (the tree is empty)";
496                 std::cout << "\n  key " << k << (found ? " is" : " is not")
497                           << " in the tree ("
498                           << plural(static_cast<int>(path.size()), "node visited",
499                                     "nodes visited")
500                           << ")\n";
501             } else {
502                 if (t.remove(k)) {
503                     std::cout << "Deleted key " << k << ".\n";
504                     t.print();
505                 } else {
506                     std::cout << "Key " << k << " is not in the tree.\n";
507                 }
508             }
509         } else if (o == 2) {
510             if (!readLine("Keys (separated by spaces or commas): ", line)) break;
511             std::vector<int> v;
512             if (!parseInts(line, v) || v.empty()) {
513                 std::cout << "Invalid list: enter integers.\n";
514                 continue;
515             }
516             int added = 0;
517             for (int x : v) {
518                 if (t.insert(x)) added++;
519                 else std::cout << "  key " << x << " is already there\n";
520             }
521             std::cout << "Inserted " << plural(added, "key", "keys") << ".\n";
522             t.print();
523         } else if (o == 5) {
524             t.print();
525         } else if (o == 6) {
526             printList(t.keys());
527         } else if (o == 7) {
528             int a = 0, b = 0;
529             if (!readInt("Left end a: ", a, eof)) { if (eof) break; continue; }
530             if (!readInt("Right end b: ", b, eof)) { if (eof) break; continue; }
531             if (a > b) { int x = a; a = b; b = x; }
532             printList(t.range(a, b));
533         } else if (o == 8) {
534             std::string why;
535             if (t.check(why))
536                 std::cout << "The tree has all the properties of a B-tree.\n";
537             else
538                 std::cout << "Property broken: " << why << ".\n";
539         } else if (o == 9) {
540             t.clear();
541             std::cout << "The tree is now empty.\n";
542         }
543     }
544     std::cout << "Goodbye!\n";
545     return 0;
546 }
```

9 References

[1] R. Bayer, E. McCreight, “Organization and maintenance of large ordered indexes”, *Acta Informatica*, vol. 1, no. 3, pp. 173–189, 1972.

[2] D. Comer, “The Ubiquitous B-Tree”, *ACM Computing Surveys*, vol. 11, no. 2, pp. 121–137, 1979.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, section 6.2.4, “Multiway Trees”.

[4] N. Wirth, *Algorithms + Data Structures = Programs*, Prentice-Hall, 1976, section 4.5, “Multiway Trees”.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 18, “B-Trees”.

[6] The ADS repository, `Trees/B Trees`, https://github.com/mihaescu/ADS.